- `Text` → Displays simple text inside the UI.

c. Understanding Each Widget in the Tree

Widget	Function	Example
MaterialApp	The root widget that sets up themes & navigation	`MaterialApp(home: Scaffold(...))`
Scaffold	Provides the basic screen layout (AppBar, body, etc.)	`Scaffold(appBar: ..., body: ...)`
AppBar	Displays a top navigation bar	`AppBar(title: Text('My App'))`
Center	Centers its child widget inside the screen	`Center(child: Text('Hello'))`
Text	Displays simple text	`Text('Hello, world!')`

- Each widget in Flutter has a specific function and can nest inside other widgets.
- This nesting structure helps organize the UI efficiently.

Step 4 : Understanding the Class and extends

a. What is a Class in Flutter?
- A class is like a blueprint for creating objects (in Flutter, widgets).
- Flutter apps are built using custom widgets, which are defined as classes.
- Each widget class defines how the UI looks and behaves.
- This is how usually we defining a widget class in Flutter :

```
class MyApp extends StatelessWidget {
```

- Explanation:
  - class MyApp → Defines a new widget called MyApp.
  - extends StatelessWidget → MyApp is based on StatelessWidget, meaning it does not change dynamically.

b. What Does extends Mean in Flutter?
- The extends keyword creates a subclass (child class) of another class.
- In Flutter, extends StatelessWidget means the widget is immutable (does not change state).
- A widget only updates when its parent widget rebuilds it.
- This is an example of full Stateless Widget :

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('My Flutter App')),
        body: Center(child: Text('Hello, Flutter!')),
      ),
    );
  }
}
```

- Explanation:
  - class MyApp extends StatelessWidget → MyApp does not have a state.
  - `build()` method → Returns the UI structure of the widget.
  - MaterialApp → The root widget of the app.

c. What is `super.key` and Why Is It Used?
- `super.key` passes the key value to the parent class (StatelessWidget or StatefulWidget).
- It helps Flutter efficiently identify and update widgets in the widget tree.
- It's optional, but recommended for better widget optimization and reusability.

d. Comparison with Java
- Both Dart and Java use extends to create a subclass.
- In Java, UI is often defined separately, while in Flutter, UI is created directly within the widget class.

```
public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

- Dart is more concise and optimized for Flutter's widget-based design.

Step 5 : Overriding Methods with `@override`

a. What is `@override` in Flutter?
- `@override` is used when a method in a child class replaces a method from its parent class.
- In Flutter, `@override` is often used in `build()`, which is inherited from StatelessWidget or StatefulWidget.
- It ensures that the method correctly overrides the parent method.
- This is an example overriding `build()` methods :

```
@override
 Widget build(BuildContext context) {
```

- Explanation:
  - `@override` → Tells Flutter that this `build()` method replaces the parent's `build()` method.
  - `build(BuildContext context)` → Defines how the UI should be displayed.
  - Without `@override`, Dart will still work, but using it improves code clarity and helps avoid mistakes.

b. Why is `@override` Important?
- Ensures that the method is correctly overridden → If the method name or signature is incorrect, Dart will show an error.
- Makes the code easier to understand → It explicitly indicates that this method is replacing a parent method.